

Model Predictive Control for Rocket Booster Recovery and Landing

Alec Weedman and Jonah Wilkes

Department of Mechanical, Materials, and Aerospace Engineering

Illinois Institute of Technology

Chicago, IL, USA

aweedman@hawk.illinoistech.edu, jwilkes@hawk.illinoistech.edu

Abstract—This project develops and tests a model predictive controller (MPC) for the powered landing of a reusable rocket booster. The booster is modeled as a simplified 3D point mass with gravity, drag, changing mass, finite propellant, thrust limits, and thrust-vector limits. The main controller is a constrained finite-horizon nonlinear model predictive controller (NMPC). A proportional-derivative (PD) controller is used as a baseline comparison. Both controllers are tested in closed-loop simulation for a nominal case and a wind-disturbed case. In the nominal case, the MPC met the cutoff condition at $t = 134.0$ s, with a final position error of 1.453 m, final horizontal error of 0.004 m, and final speed of 0.122 m/s. The nominal PD controller did not meet the cutoff condition and ended with a final position error of 148.734 m. Under wind disturbance, the MPC ended with a final position error of 8.294 m and final speed of 0.502 m/s, while the PD baseline ended with a final position error of 162.144 m. These results show that MPC is better suited for this constrained landing problem than the simpler PD baseline, with room to improve.

Index Terms—model predictive control, optimal control, rocket landing, constrained control

I. INTRODUCTION

Reusable rocket landing is a challenging control problem as the landing vehicle must balance slowing down, moving toward a target, saving fuel, and staying within actuator limits throughout descent. During descent, the controller must choose thrust commands that reduce position and velocity errors while respecting thrust magnitude and direction limitations, ground, and propellant constraints.

This makes booster recovery a good application for optimization based control. Prior landing studies establish powered descent as an optimal control problem with state, input, and terminal constraints [1], [2]. Precise landing is also important for reusable rockets as accurate descent trajectories allow boosters to be safely landed, refueled, and reused [3].

The goal of this project is to compare a constrained model predictive controller (MPC) against a simpler PD baseline controller for a rocket booster landing simulation. The MPC controller plans over a short future horizon and updates its plan at every time step. The PD controller follows a simple commanded reference trajectory. The comparison uses final landing error, final speed, fuel and thrust use, and landing time, along with a wind disturbance robustness study.

II. SYSTEM MODEL

The booster itself is modeled as a simple three dimensional point mass. The state is

$$x = [r_x \ r_y \ r_z \ v_x \ v_y \ v_z \ m]^T, \quad (1)$$

where r_x, r_y, r_z are position, v_x, v_y, v_z are velocity, and m is mass. The control input is the thrust vector

$$T = [T_x \ T_y \ T_z]^T. \quad (2)$$

The simplified equations of motion are

$$\dot{r} = v, \quad (3)$$

$$\dot{v} = \frac{T}{m} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + a_D + a_w, \quad (4)$$

$$\dot{m} = -\frac{\|T\|}{I_{sp}g_0}. \quad (5)$$

Here, g is gravity, I_{sp} is specific impulse, a_D is drag acceleration, and a_w is the wind disturbance. This simplified model was used to keep the project focused on the landing guidance and controller comparison. Modeling the full upright booster, including rotation, actuator dynamics, and detailed aerodynamics, was beyond the scope of this project.

The atmosphere is modeled with a simple exponential density model,

$$\rho(r_z) = \rho_0 e^{-r_z/H}, \quad (6)$$

with the drag acceleration modeled as

$$a_D = -\frac{1}{2m} \rho C_D A \|v\| v. \quad (7)$$

This simple atmospheric drag model keeps the simulation manageable while still capturing the main effect of atmospheric drag.

This model does not include full attitude dynamics, such as booster rotation, actuator dynamics, or detailed aerodynamic moments. Instead, the thrust vector is directly limited by a maximum tilt angle of 20° . This assumes that a lower-level attitude controller can point the thrust vector fast enough compared with the translational motion. The simulation therefore

focuses on landing guidance rather than full vehicle attitude control.

The simulation parameters were chosen to roughly represent a Falcon 9 style booster during the powered landing phase, while still keeping the model simple. The dry mass was set to 22 000 kg with an appropriate amount of fuel, and the maximum thrust was set to 3×845 kN, representing the use of three Merlin-class engines during landing. The specific impulse was set to 280 s, which is a representative rocket engine performance value for the simplified fuel consumption model.

TABLE I
SIMULATION PARAMETERS

Parameter	Symbol	Value
Initial mass	m_0	70 000 kg
Dry mass	m_{dry}	22 000 kg
Specific impulse	I_{sp}	280 s
Gravity	g	9.81 m/s ²
Maximum thrust	T_{max}	2 535 000 N
Maximum tilt angle	θ_{max}	20°
Atmospheric density	ρ_0	1.225 kg/m ³
Scale height	H	8500 m
Drag area parameter	$C_D A$	10 m ²
Time step	Δt	1 s
MPC horizon length	N	20 steps
Prediction horizon	T_p	20 s
Simulation time	t_f	180 s

The initial state is

$$x(0) = [2500 \quad -1200 \quad 30000 \quad -20 \quad 10 \quad -250 \quad 70000]^T. \quad (8)$$

And the target is

$$r_f = [0 \quad 0 \quad 0]^T, \quad v_f = [0 \quad 0 \quad 0]^T. \quad (9)$$

III. PROBLEM FORMULATION

The objective of this control problem is to land near the target with a low final speed while staying within thrust, engine tilt, altitude, and mass limitations. The MPC does this by solving a short horizon optimization problem at every time step.

Since the target position and velocity were defined as zero, the error vector can be written as the current position and velocity state:

$$e_k = [r_{x,k} \quad r_{y,k} \quad r_{z,k} \quad v_{x,k} \quad v_{y,k} \quad v_{z,k}]^T. \quad (10)$$

The MPC cost is

$$J = \sum_{k=0}^{N-1} (e_k^T Q e_k + R_{\text{cmd}} \|c_k\|^2 + R_{\text{fuel}} \rho_k^2) \Delta t + e_N^T P e_N. \quad (11)$$

The running cost penalizes position error, velocity error, command input, and fuel use. The state weighting matrix was selected as

$$Q = \text{diag}(0.005, 0.005, 0.03, 0.25, 0.25, 3.0), \quad (12)$$

where the entries correspond to

$$[r_x \quad r_y \quad r_z \quad v_x \quad v_y \quad v_z]^T.$$

The larger weights on velocity, especially vertical velocity, encourage the booster to reduce speed during descent. The terminal cost uses

$$P = \text{diag}(500, 500, 1000, 1000, 1000, 3000), \quad (13)$$

which more strongly penalizes final position and velocity error at the end of the prediction horizon. The larger terminal weights help drive the predicted final state toward the landing target.

The optimizer command is

$$c_k = [\rho_k \quad p_{x,k} \quad p_{y,k}]^T, \quad (14)$$

where ρ_k controls thrust magnitude and $p_{x,k}, p_{y,k}$ control thrust direction. This command is converted into a thrust vector that always satisfies

$$0 \leq \|T_k\| \leq T_{\text{max}}, \quad (15)$$

$$\theta_k \leq \theta_{\text{max}}, \quad (16)$$

$$m_k \geq m_{\text{dry}}, \quad (17)$$

$$r_{z,k} \geq 0. \quad (18)$$

This setup ensures that the applied thrust commands are always checked against the physical limits of the model. The thrust magnitude and tilt limits are enforced directly through the thrust parameterization, while ground and dry-mass violations are discouraged using large soft penalty weights in the cost function.

An engine cutoff condition is also used so that the simulation stops once the booster reaches a successful near-touchdown state. The engine shuts off when

$$\sqrt{r_x^2 + r_y^2} \leq 3 \text{ m}, \quad (19)$$

$$r_z \leq 1.5 \text{ m}, \quad (20)$$

$$\|v\| \leq 1 \text{ m/s}. \quad (21)$$

These thresholds represent a successful near-touchdown condition for the simplified landing model, and were chosen arbitrarily.

IV. METHODS

A. Model Predictive Controller

The MPC controller solves an optimization problem over a 20 second horizon. At each time step, it predicts the future booster motion, finds a set of thrust commands, applies the first command, and repeats the process at the next time step. This gives the controller feedback, as each new optimization starts from the current state of the booster.

The prediction model uses a forward Euler integration approach with $\Delta t = 1$ s. The optimizer is solved in Python through `scipy.optimize.minimize`. This method was used because it supports simple bounds on the optimization variables. The optimizer used a maximum of 50 iterations and

a function tolerance of 10^{-4} . The previous solution is shifted forward and used as the next initial guess, which helps reduce solve time, but also makes the closed-loop result dependent on the sequence of previous optimizer solutions.

B. PD Baseline Controller

The baseline controller is a PD controller that follows a planned reference trajectory. A simple reference path was generated between the initial position and the landing target because a direct PD controller, using only the target point as the reference, was not a meaningful comparison to the MPC. Without the planned trajectory, the PD controller was unable to produce a competitive landing response. Adding the reference path gave the PD baseline a reasonable tracking problem for comparison. The controller computes a desired acceleration using

$$a_{\text{cmd}} = K_p(r_{\text{ref}} - r) + K_d(v_{\text{ref}} - v) + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}. \quad (22)$$

The proportional and derivative gains were

$$K_p = \begin{bmatrix} 0.004 & 0.004 & 0.003 \end{bmatrix}, \quad (23)$$

$$K_d = \begin{bmatrix} 0.08 & 0.08 & 0.70 \end{bmatrix}. \quad (24)$$

These gains were tuned manually to give a reasonable reference-tracking response while keeping the controller within the same thrust, tilt, and fuel limits used by the MPC.

The thrust command is then

$$T = ma_{\text{cmd}}. \quad (25)$$

The same thrust magnitude, engine tilt, and fuel limits from the MPC are applied to the PD controller. This ensures the comparison works within the same constraints.

C. Simulation Cases

Two cases are simulated. The nominal case has no wind:

$$a_w = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T. \quad (26)$$

The wind-disturbed case uses a constant lateral acceleration:

$$a_w = \begin{bmatrix} 0.10 & -0.05 & 0 \end{bmatrix}^T \text{ m/s}^2. \quad (27)$$

This is a simple way to represent wind or unmodeled external influences on the system.

For each case, the simulation records position, velocity, mass, thrust, engine tilt angle, final error, final speed, fuel use, and thrust effort.

V. RESULTS

A. Nominal Closed-Loop Response

The nominal case compares MPC and PD control without wind. Fig. 1 shows the position response, Fig. 2 shows the velocity response, and Fig. 3 shows the three-dimensional landing path.

The MPC met the engine cutoff condition at $t = 134.0$ s. At cutoff, the final position error was 1.453 m, the final horizontal

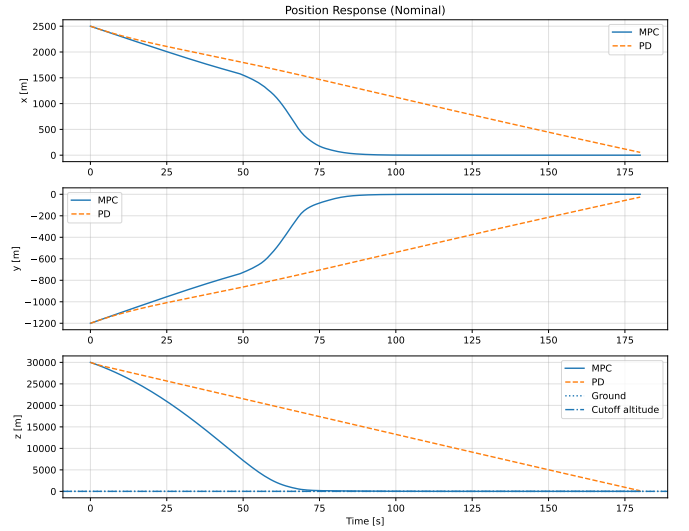


Fig. 1. Position response for the MPC and PD controllers. The MPC reaches the terminal landing condition, while the PD baseline remains above the target altitude.

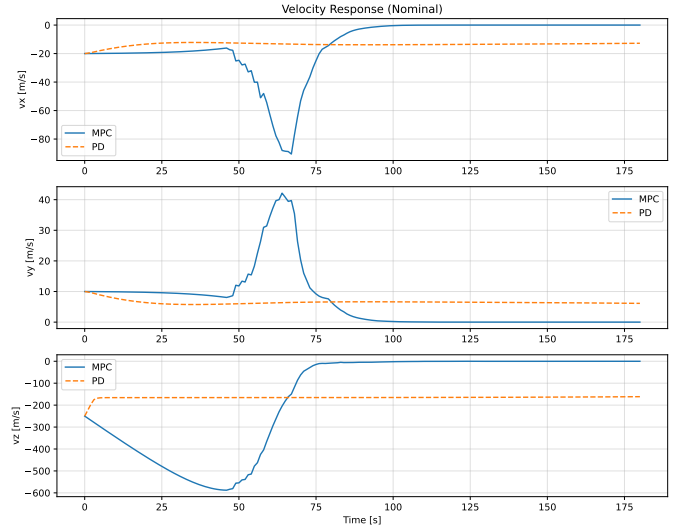


Fig. 2. Velocity response for the MPC and PD controllers. The MPC drives the velocity components close to zero before cutoff, while the PD baseline keeps a large final velocity.

error was 0.004 m, and the final speed was 0.122 m/s. This is a successful soft-landing result for the simplified model.

The PD baseline did not meet the cutoff condition. It ended with a final position error of 148.734 m, a final horizontal error of 60.532 m, a final altitude of 135.860 m, and a final speed of 162.090 m/s. This means the PD controller did not complete a soft landing in the same simulation time.

Across these simulations, no thrust, tilt, ground, or mass constraint violations occurred for either controller, both methods remained feasible with respect to the imposed constraints, although only the MPC achieved the engine cutoff condition.

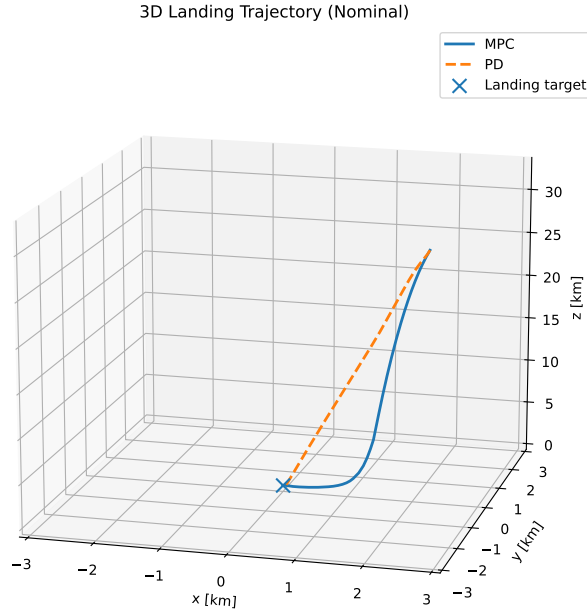


Fig. 3. 3D landing trajectory. The MPC reaches the landing target and triggers cutoff, while the PD baseline does not.

TABLE II
NOMINAL CLOSED-LOOP PERFORMANCE

Metric	MPC	PD Baseline
Final position error [m]	1.453	148.734
Final horizontal error [m]	0.004	60.532
Final altitude [m]	1.453	135.860
Final speed [m/s]	0.122	162.090
Cutoff condition met	True	False
Fuel used [kg]	26124.075	31891.998
Final mass [kg]	43875.925	38108.002
Fuel margin above dry [kg]	21875.925	16108.002
Thrust integral [Ns]	71757608.171	87600941.002
Maximum thrust [N]	2532735.227	2535000.000
Maximum tilt angle [deg]	20.000	3.280
Minimum altitude [m]	1.453	135.860
Minimum mass [kg]	43875.925	38108.002

B. Nominal Control Effort

Fig. 4 shows the nominal thrust magnitude, and Fig. 5 shows the individual thrust components.

The MPC used more lateral thrust than the PD controller, reaching the 20° tilt limit. This helped remove horizontal error. The PD controller only reached a maximum tilt angle of 3.280° , which was not enough to correct the trajectory before the end of the simulation. In this ideal case, the MPC also used less total fuel than the PD controller.

C. Computational Performance

The MPC controller requires an optimization solve at each control update, while the PD controller does not. This makes MPC much more computationally expensive, but it also allows the controller to plan ahead and handle constraints.

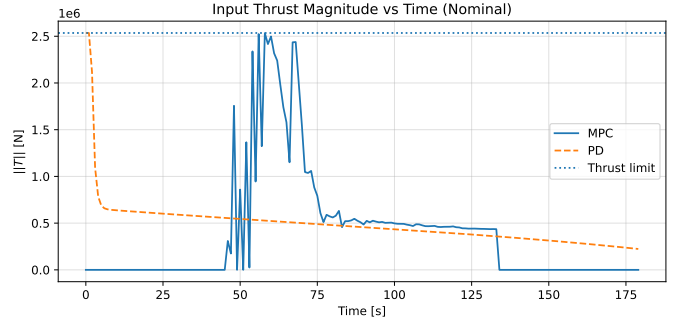


Fig. 4. Thrust magnitude history. Both controllers remain within the maximum thrust limit.

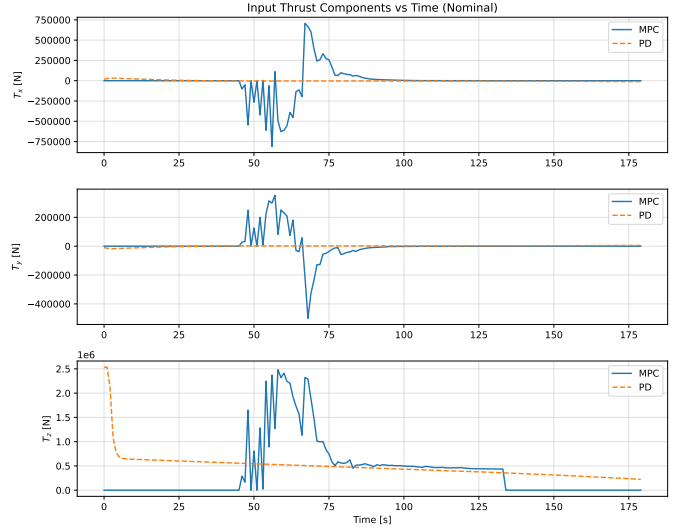


Fig. 5. Thrust component history. The MPC uses stronger lateral thrust corrections while also managing the vertical thrust needed for descent and landing.

In the nominal case, the MPC stopped after 134 iterations because it reached the cutoff condition. In the wind case, it ran for the full 180 steps. These solve times are acceptable for this simulation study, but they are not fast enough for in-flight, real-time controllers. The optimizer was also not perfectly consistent between runs. The nominal result reported here represents a good solve, where the cutoff condition was reached at $t = 134.0$ s. In other runs with the same general setup, the nominal MPC case often reached cutoff closer to $t \approx 155$ s. This variation is likely due to the nonlinear and nonconvex nature of the finite-horizon optimization problem,

TABLE III
MPC COMPUTATIONAL PERFORMANCE

Metric	Nominal MPC	Wind MPC
Number of MPC solves	134	180
Mean solve time [s]	3.1785	5.1636
Max solve time [s]	16.5180	23.9258
Optimizer success [%]	82.1	88.3

the finite iteration limit used in the optimizer, and the dependence of each solve on the shifted previous solution used as the next initial guess. Therefore, the reported results should be interpreted as one representation of a closed-loop simulation rather than a perfectly repeatable global solution. A practical implementation would need faster solvers, a shorter horizon, improved initialization, or a convex optimization approach, as described in previous studies [1].

VI. ROBUSTNESS STUDY

Next, a wind influenced case was simulated to test the controllers under unknown perturbed conditions. The wind is modeled as a constant lateral acceleration imparted on the simulated particle, as there were no aerodynamic simulations. This is simpler than a full atmospheric model, but it is enough to test whether the controllers can adapt to a persistent horizontal disturbance.

The wind influenced position, velocity, and 3D trajectory responses are shown in Fig. 6, Fig. 7, and Fig. 8.

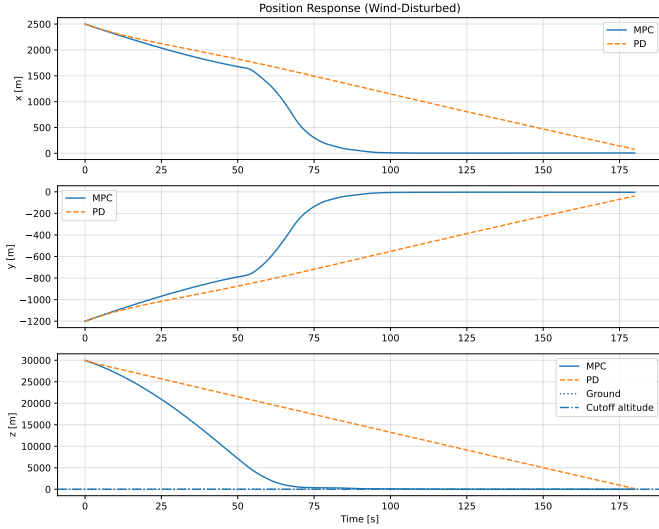


Fig. 6. Wind-disturbed position response. The MPC remains near the landing target, while the PD baseline remains relatively far from the desired terminal state.

The wind-disturbed MPC did not meet the cutoff condition, but it still reached a low speed near-landing state. Its final position error was 8.294 m, final horizontal error was 7.988 m, final altitude was 2.233 m, and final speed was 0.502 m/s. The cutoff condition was not met because the final horizontal error and altitude were still above their limits.

The wind-disturbed PD baseline performed much worse. It ended with a final position error of 162.144 m, final horizontal error of 88.485 m, final altitude of 135.872 m, and final speed of 162.089 m/s.

The wind-disturbed thrust magnitude and thrust components are shown in Fig. 9 and Fig. 10.

The wind case shows that the MPC is affected by the lateral disturbance, but it still performs much better than the PD baseline. Compared with the nominal MPC result, the

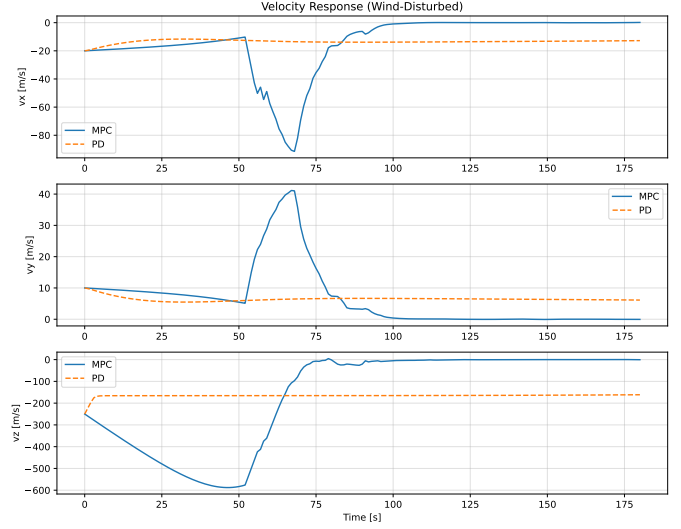


Fig. 7. Wind-disturbed velocity response. The MPC still reduces the final velocity components to small values, while the PD baseline keeps a large final velocity.

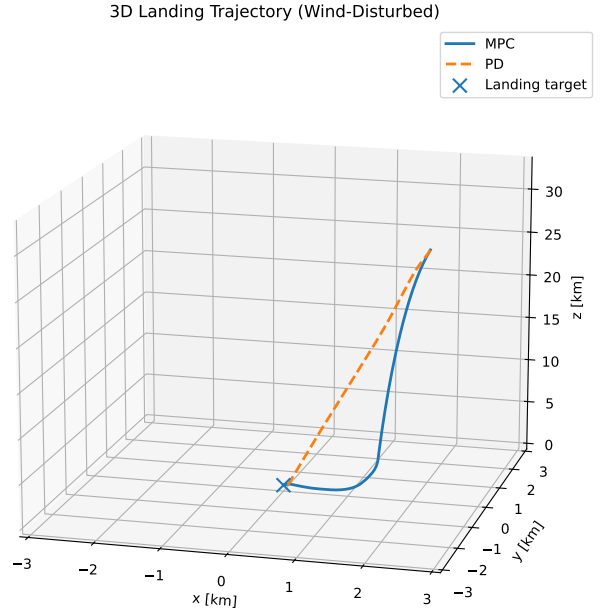


Fig. 8. Wind-disturbed 3D landing trajectory. The MPC reaches a low speed near-landing state, while the PD baseline does not.

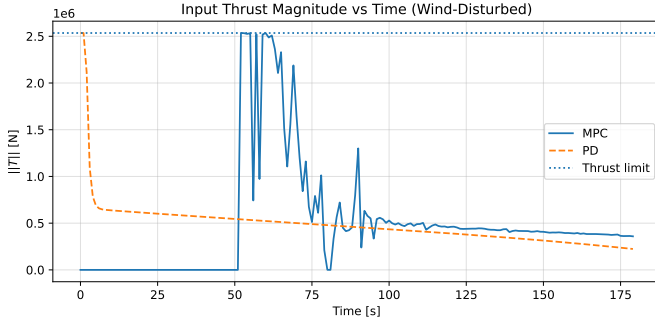


Fig. 9. Wind-disturbed thrust magnitude history. Both controllers remain within the maximum thrust limit.

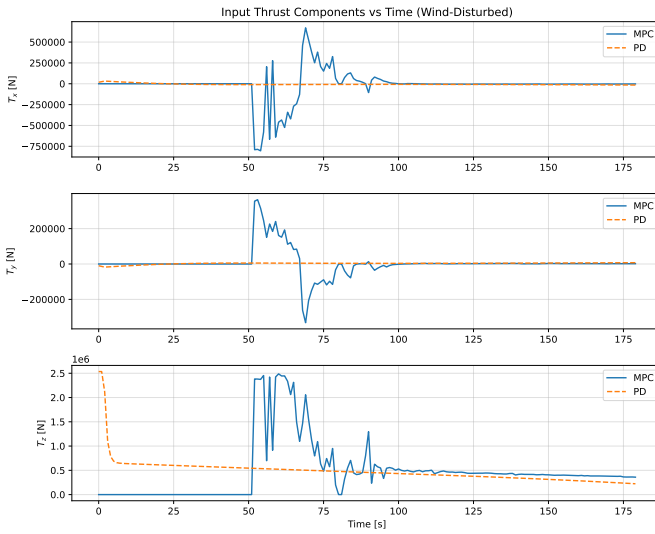


Fig. 10. Wind-disturbed thrust component history. The MPC uses stronger lateral thrust commands to counter the wind disturbance.

TABLE IV
WIND-DISTURBED CLOSED-LOOP PERFORMANCE

Metric	MPC Wind Case	PD Wind Case
Final position error [m]	8.294	162.144
Final horizontal error [m]	7.988	88.485
Final altitude [m]	2.233	135.872
Final speed [m/s]	0.502	162.089
Cutoff condition met	False	False
Fuel used [kg]	32850.517	31896.447
Final mass [kg]	37149.483	38103.553
Fuel margin above dry [kg]	15149.483	16103.553
Thrust integral [N s]	90233800.163	87613159.425
Maximum thrust [N]	2535000.000	2535000.000
Maximum tilt angle [deg]	20.000	4.372
Minimum altitude [m]	1.171	135.872
Minimum mass [kg]	37149.483	38103.553

final position error increased from 1.453 m to 8.294 m. Fuel use also increased from 26 124.075 kg to 32 850.517 kg. The final speed remained low, increasing only from 0.122 m/s to 0.502 m/s. Therefore, the main wind effect was on horizontal position error.

VII. DISCUSSION

The nominal results show that MPC performed much better than the PD control. The MPC reached the cutoff condition at $t = 134.0$ s, with a final position error of 1.453 m and final speed of 0.122 m/s. The PD controller did not reach the cutoff condition and ended far from the target with a high final speed.

The MPC also used less fuel in the nominal case. The MPC used 26 124.075 kg, while the PD controller used 31 891.998 kg. This means the MPC used about 5767.923 kg less propellant while also achieving better landing accuracy.

The wind case shows the main limitation of the current MPC tuning. The controller kept the vehicle slow and near the target, but the final horizontal error was still too large to trigger cutoff. This suggests that the MPC remained stable, but it needed more lateral correction under wind. Possible improvements include increasing the horizontal error penalties or increasing prediction horizon length.

The PD baseline is useful as a simple comparison method. It is easy to implement and very fast to compute. However, it does not plan ahead, and does not optimize fuel use. This is why it stayed within the given constraints but still failed to land accurately.

The largest drawback of MPC is computation time. The current Python implementation is not fast enough for a real onboard controller. However, it is sufficient for this project because the goal is to show the structure and behavior of constrained optimization-based control. A flight-ready version would need faster solvers, reduced update intervals, or a convex/successive-convexification method [1].

This model also has important simplifications. The booster is treated as a point mass, which means attitude dynamics are not modeled, drag is simplified, and wind is represented as a constant acceleration. These choices keep the project manageable, but limit the physical realism.

VIII. CONCLUSION

This optimization project implemented and evaluated a MPC controller for a simplified reusable rocket booster landing problem. The booster model included gravity, simple drag, variable mass via fuel loss, thrust magnitude limits, and engine tilt limits. A PD controller was implemented for comparison.

In the nominal case, the MPC met the cutoff condition at $t = 134.0$ s, with a final position error of 1.453 m, final horizontal error of 0.004 m, and final speed of 0.122 m/s. The PD controller did not meet the cutoff condition and ended with much larger position and speed errors. The MPC also used less fuel than the PD controller in the nominal case.

In the wind case, the MPC did not meet the cutoff condition, but it still ended near the target with a low final speed. The wind influenced MPC ended with a final position error of

8.294 m and final speed of 0.502 m/s. The PD controller ended with a final position error of 162.144 m and final speed of 162.089 m/s. This shows that MPC handled the disturbance much better than the PD baseline, even though the wind case still needs improved tuning. For further robustness validation, running the simulation at multiple different initial states could prove the optimization model more effective. This would be involved in future work.

Overall, the MPC controller satisfied the goals of this project as it showed the ability to plan ahead, handle constraints, reduce terminal error via cost functions, and improved fuel usage over the PD controller. In the future, this modeling should focus on faster computation methods, better wind modeling, attitude and inertial simulation of the craft's dynamics, and more realistic aerodynamic and engine models.

CONTRIBUTION STATEMENT

Alec Weedman and Jonah Wilkes contributed equally to the project concept, setup, simulation, controller comparison, and report preparation. The implementation included the simplified rocket landing model, the MPC controller, the PD baseline, closed-loop simulation, result plotting, and performance comparison.

AI ASSISTANCE STATEMENT

ChatGPT was used as an assistance tool during this project. It helped with parts of code revision, report organization, wording, and explanation of the methods and results. The code, simulations, figures, numerical results, interpretation, and report were completed and reviewed by the authors, with ChatGPT used as a support tool during revision and editing. The final submitted work represents the authors' own work.

REFERENCES

- [1] B. Açıkmeşe, J. M. C. III, and L. Blackmore, "Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem," *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2104–2113, 2013.
- [2] A. Botelho, M. Martinez, C. Recupero, A. Fabrizi, and G. D. Zaiacomo, "Design of the landing guidance for the retro-propulsive vertical landing of a reusable rocket stage," *CEAS Space Journal*, vol. 14, pp. 551–564, 2022.
- [3] L. Blackmore, "Autonomous precision landing of space rockets," in *Frontiers of Engineering: Reports on Leading-Edge Engineering from the 2016 Symposium*. National Academies Press, 2016.